EquiDados | Índices da Etapa 04

POSTGRESQL · NEON · 08 NOV 2026

Definição, campos, justificativa e consultas beneficiadas

Índice 1 — equipe + data

idx_mock_atendimentos_equipe_data_etapa04

Chave composta (codigo_equipe, data_atendimento). A equipe é comparada por igualdade; dentro de cada equipe, a consulta restringe o período por intervalo de datas. Essa ordem atende o JOIN e a janela temporal juntos.

```
CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_equipe_data_etapa04
    ON mock_pec_atendimentos (codigo_equipe, data_atendimento);
```

PREDICADO: EQUIPE = ? AND DATA BETWEEN ? AND ?

Índice 2 — data + distrito

idx_mock_atendimentos_data_distrito_etapa04

Chave composta (data_atendimento, distrito_sanitario_codigo). A janela recente restringe primeiro a data; o índice também contém o distrito selecionado e agregado no resultado territorial.

```
CREATE INDEX IF NOT EXISTS idx_mock_atendimentos_data_distrito_etapa04
    ON mock_pec_atendimentos (data_atendimento, distrito_sanitario_codigo);
```

PREDICADO: DATA BETWEEN ? AND ? · DISTRITO IS NOT NULL

Por que compostos? Os índices básicos da aplicação cobrem equipe, data e distrito isoladamente. Estes dois foram adicionados para os padrões conjuntos observados nas consultas: igualdade da equipe mais faixa de data e faixa de data mais leitura do distrito. A escolha de colunas e ordem segue esses predicados, não apenas a existência genérica dos campos.

Origem: definições presentes em `Etapa_04_Consultas_Avancadas.sql`. Os registros consultados são da tabela `mock_pec_atendimentos`, explicitamente identificada como fictícia/simulada no projeto.

Uso nas consultas e planos EXPLAIN

PLANOS EXPLAIN SEM ANALYZE · SOMENTE LEITURA

Correspondência com filtros/joins e verificação no PostgreSQL/Neon

Índice 1 → Consulta 1: anti join

A junção por equipe e a faixa móvel de 90 dias aparecem no predicado do LEFT JOIN. O anti join seleciona as equipes sem correspondência.

```
LEFT JOIN mock_pec_atendimentos a
    ON a.codigo_equipe = e.codigo_equipe
   AND a.data_atendimento >= CURRENT_DATE - INTERVAL '90 days'
   AND a.data_atendimento <= CURRENT_DATE
WHERE a.id_atendimento IS NULL
```

```
Bitmap Index Scan on idx_mock_atendimentos_equipe_data_etapa04
Index Cond: ((codigo_equipe = e.codigo_equipe)
  AND (data_atendimento >= CURRENT_DATE - '90 days')
  AND (data_atendimento <= CURRENT_DATE))
```

Índice 2 → visão territorial e Consulta 2

`vw_equidade_distrital` agrega atendimentos dos últimos 90 dias por distrito. A CTE `atendimentos_por_distrito` da consulta FULL OUTER JOIN repete o mesmo filtro temporal/territorial.

```
WHERE data_atendimento >= CURRENT_DATE - INTERVAL '90 days'
  AND data_atendimento < CURRENT_DATE + INTERVAL '1 day'
  AND distrito_sanitario_codigo IS NOT NULL
GROUP BY distrito_sanitario_codigo
```

```
Index Only Scan using idx_mock_atendimentos_data_distrito_etapa04
Index Cond: ((data_atendimento >= CURRENT_DATE - '90 days')
  AND (data_atendimento < CURRENT_DATE + '1 day')
  AND (distrito_sanitario_codigo IS NOT NULL))
```

Verificação: os planos foram obtidos com `EXPLAIN` no PostgreSQL/Neon configurado para o projeto. O plano do anti join selecionou `Bitmap Index Scan` pelo índice equipe/data; o agregado territorial selecionou `Index Only Scan` pelo índice data/distrito. `EXPLAIN` verifica a decisão do planejador, sem executar a consulta nem medir o tempo real.

Nota de execução: o PostgreSQL decide o plano conforme estatísticas, volume e parâmetros de cada banco. O mapeamento mostra que os índices correspondem diretamente aos predicados implementados; a seleção observada neste ambiente confirma o uso nas consultas representativas da Etapa 04.